

Final Project (Screenshots and Written Portions)

In []: Authors: Valerie De La Fuente, Sam Caruthers, Maggie Truong, Anushka Larson

1. Data in BigQuery

1.2 Shell Script in Cloud Shell

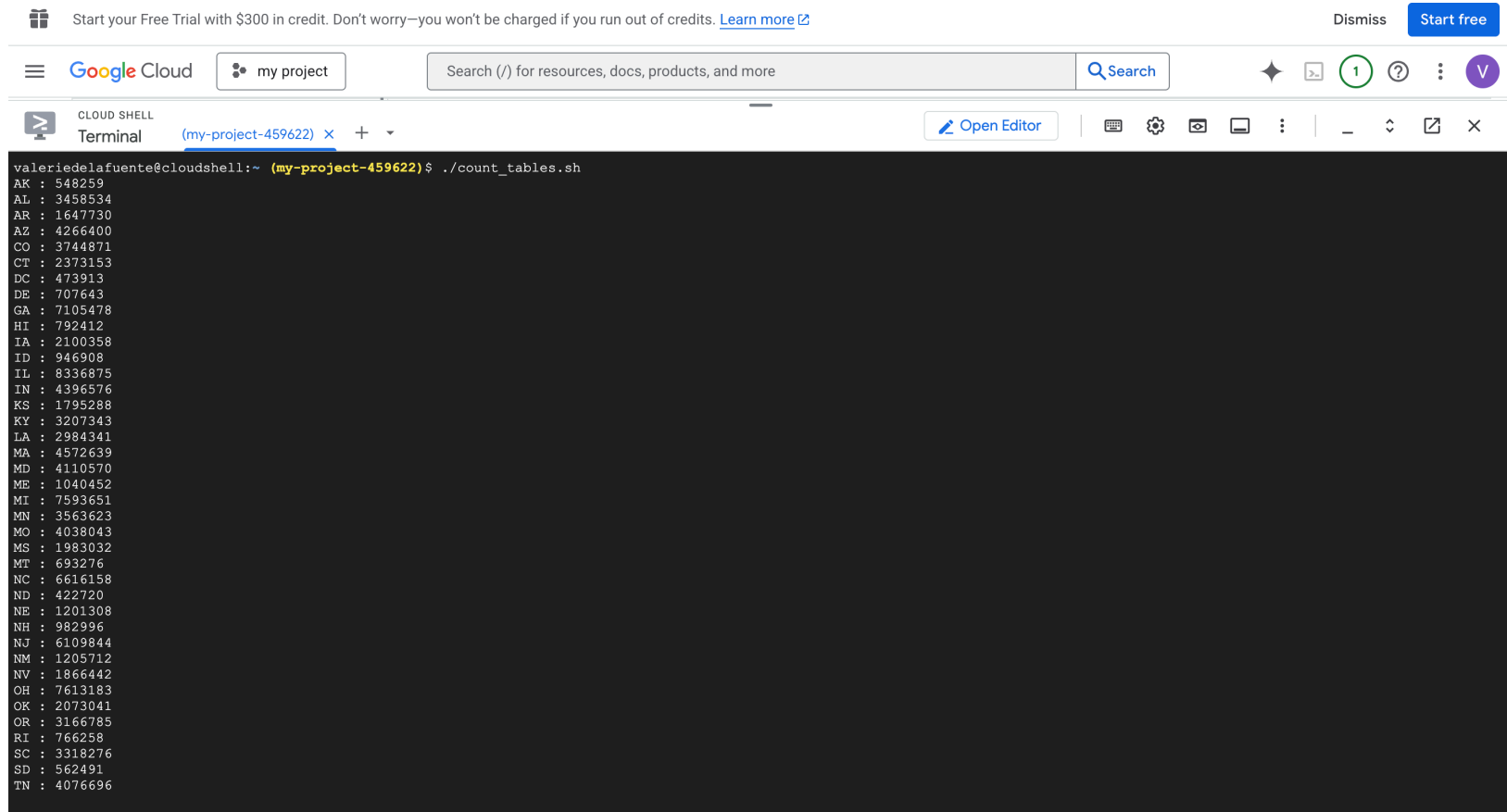
The following is the code that we used in our `count_tables.sh` script:

```
states=(
AK AL AR AZ CO CT DC DE GA HI IA ID IL IN KS KY LA MA MD ME
MI MN MO MS MT NC ND NE NH NJ NM NV OH OK OR RI SC SD TN UT
VA VT WA WI WV WY
)
for state in "${states[@]"; do
    count=$(
        bq query \
        --quiet \
        --format=csv \
        --use_legacy_sql=false \
        "SELECT COUNT(*) FROM `pstat-135-235-spring-2025.voterfile.${state}`" \
        | tail -n +2
    )
    if [[ -z "$count" ]]; then
        echo "$state : <error or no such table>"
    else
        echo "$state : ${count//[[:space:]]/}"
    fi
done
```

To extract the rows in each table, we ran these two lines of code:

```
chmod +x count_tables.sh
./count_tables.sh
```

This is the **output** of those two lines of code:



The screenshot shows a Google Cloud Shell interface. At the top, there's a banner for a free trial. Below it, the Google Cloud logo and 'my project' are visible. A search bar is present. The main terminal window shows the command `./count_tables.sh` being executed, resulting in a list of 50 US states and their corresponding voter record counts.

```
valeriedelafuente@cloudshell:~ (my-project-459622)$ ./count_tables.sh
AK : 548259
AL : 3458534
AR : 1647730
AZ : 4266400
CO : 3744871
CT : 2373153
DC : 473913
DE : 707643
GA : 7105478
HI : 792412
IA : 2100358
ID : 946908
IL : 8336875
IN : 4396576
KS : 1795288
KY : 3207343
LA : 2984341
MA : 4572639
MD : 4110570
ME : 1040452
MI : 7593651
MN : 3563623
MO : 4038043
MS : 1983032
MT : 693276
NC : 6616158
ND : 422720
NE : 1201308
NH : 982996
NJ : 6109844
NM : 1205712
NV : 1866442
OH : 7613183
OK : 2073041
OR : 3166785
RI : 766258
SC : 3318276
SD : 562491
TN : 4076696
```

Explanation of Shell Script: Our shell script looped through all of 50 United States state abbreviations. It queried each matching voter file table in BigQuery. For each of the 50 states, our shell script ran an SQL `SELECT COUNT(*)` query. This counted the number of voter records that existed for that particular state. To send the query, the bq command-line tool was

utilized. Finally, the result was cleaned using string and tail formatting. In the event of an error or a missing table, the script prints a message that indicates such an event. Otherwise, the state code is printed with the number of rows in the table.

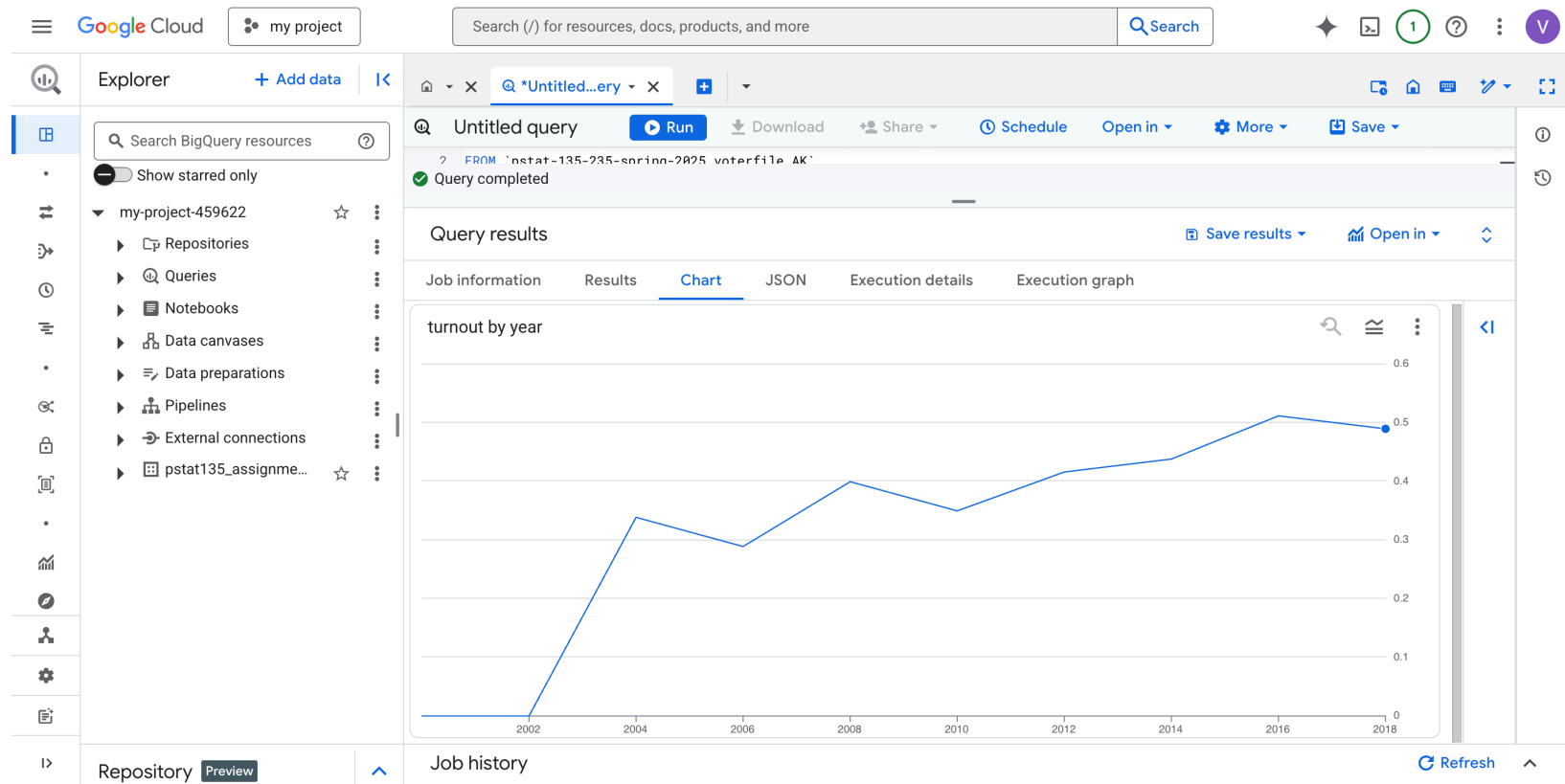
1.3 Voter Turnout Query

The following is the resulting query from ChatGPT:

```
SELECT '2000' AS year, COUNTIF(General_2000 = 'Y') / COUNT(*) AS turnout
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2002', COUNTIF(General_2002 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2004', COUNTIF(General_2004 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2006', COUNTIF(General_2006 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2008', COUNTIF(General_2008 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2010', COUNTIF(General_2010 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2012', COUNTIF(General_2012 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2014', COUNTIF(General_2014 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2016', COUNTIF(General_2016 = 'Y') / COUNT(*)
FROM `pstat-135-235-spring-2025.voterfile.AK`
UNION ALL
SELECT '2018', COUNTIF(General_2018 = 'Y') / COUNT(*)
```

```
FROM `pstat-135-235-spring-2025.voterfile.AK`
ORDER BY year;
```

This is the resulting chart:



1.3 BigQuery Notebook

The notebook is running on Vertex AI Workbench. The region is us-west4 (LV). With the "Open Job" page, we learn that each BigQuery job has detailed information available. It displays the amount of data scanned, the query text, the job's success status, and the region (which matches the voter file dataset's location). The timeline view helps analyze performance by breaking down the phases of the query's execution. This page is helpful for debugging, tracking query costs, and verifying where and how the query was executed.

The following is the code used in the notebook:

```
import bigframes.pandas as bf
df = bf.read_gbq("pstat-135-235-spring-2025.voterfile.AK")
df.head(20)

%%bigquery results
SELECT
  year,
  ROUND(SUM(IF(voted = 'Y', 1, 0)) / COUNT(*), 6) AS turnout
FROM (
  SELECT '2000' AS year, General_2000 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2002' AS year, General_2002 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2004' AS year, General_2004 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2006' AS year, General_2006 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2008' AS year, General_2008 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2010' AS year, General_2010 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2012' AS year, General_2012 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2014' AS year, General_2014 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2016' AS year, General_2016 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK` UNION ALL
  SELECT '2018' AS year, General_2018 AS voted FROM `pstat-135-235-spring-
2025.voterfile.AK`
)
GROUP BY year
ORDER BY year;

from matplotlib import pyplot as plt
import seaborn as sns
def _plot_series(series, series_name, series_index=0):
```

```

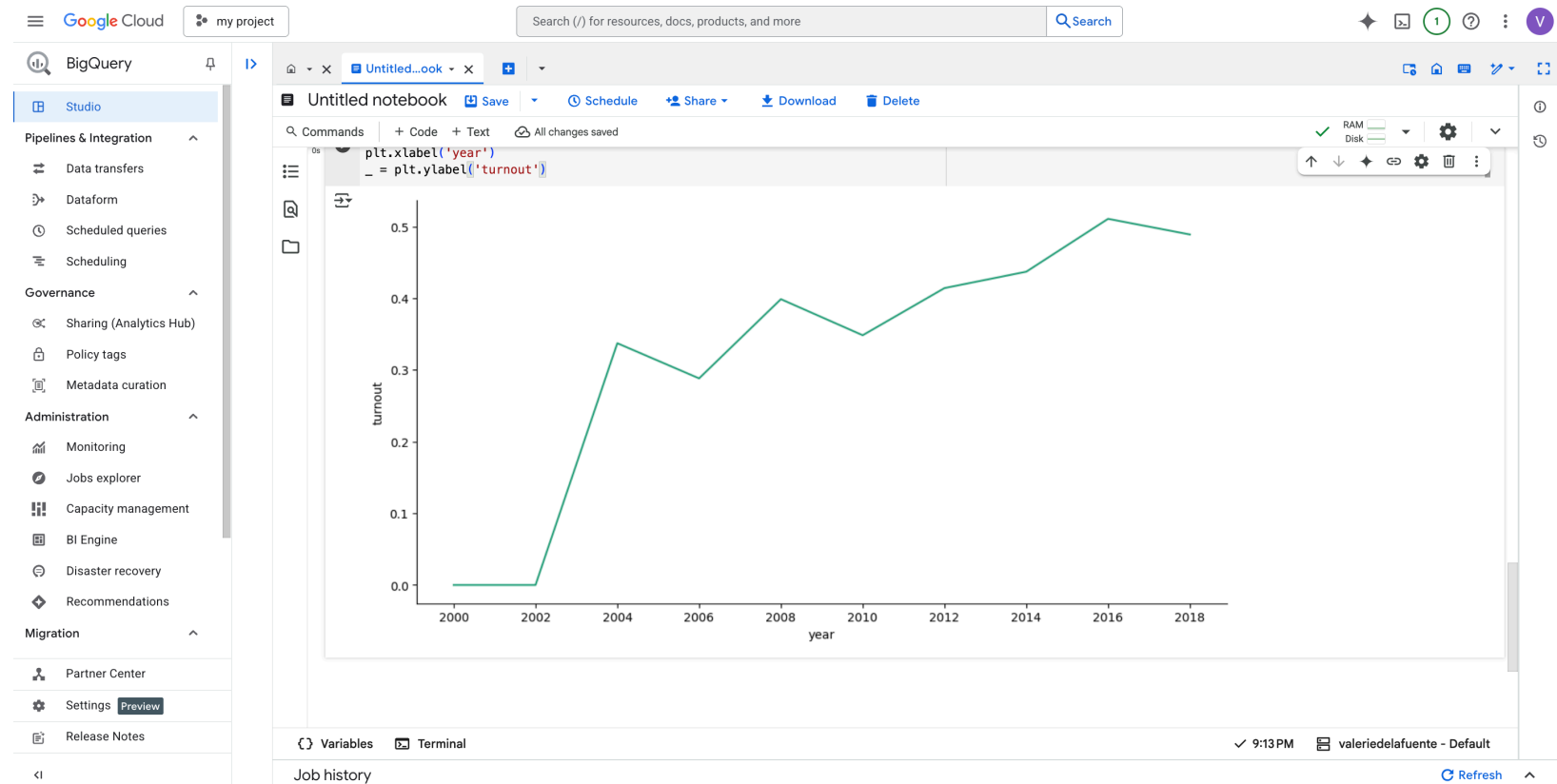
palette = list(sns.palettes.mpl_palette('Dark2'))
xs = series['year']
ys = series['turnout']

plt.plot(xs, ys, label=series_name, color=palette[series_index % len(palette)])

fig, ax = plt.subplots(figsize=(10, 5.2), layout='constrained')
df_sorted = results.sort_values('year', ascending=True)
_plot_series(df_sorted, '')
sns.despine(fig=fig, ax=ax)
plt.xlabel('year')
_ = plt.ylabel('turnout')

```

This is the resulting chart:



Pros and Cons of Each Method:

The first method has the advantage of reading tables directly into a dataframe, making it easy to manipulate data within the analysis environment. However, it can be inefficient when working with large volumes of data, leading to performance bottlenecks. The second method excels at summary queries and is well-suited for generating visualizations, but it tends to be less flexible overall and often requires writing lengthy SQL code to achieve more customized results.

2. Duplicate Data into Databricks

2.1. Choose a state and export the data to your Databricks storage location:

The screenshot shows the Databricks Catalog Explorer interface. On the left, the 'Catalog' sidebar is open, showing a tree view with 'My organization' > 'databricks_demo' > 'default' > 'voterfile-export' selected. The main panel displays the details for the 'voterfile-export' volume. The breadcrumb path is 'Catalog Explorer > databricks_demo > default > voterfile-export'. A search bar at the top contains 'Search data, notebooks, recents, and more...'. A blue button 'Upload to this volume' is visible. The 'Details' tab is active, showing a table with the following information:

Resource Name	/metastores/b6ebc2a9-7d30-44e9-8f8a-64a83c181bd0/volumes/fc98bbde-cf1d-4af1-bcfb-3368c96ea5df
Volume Type	MANAGED
Storage Location	gs://databricks-4375967546298522-unitycatalog/4375967546298522/_unitystorage/catalogs/449aad47-534b-43d8-94e7-7a0926d12de9/volumes/fc98bbde-cf1d-4af1-bcfb-3368c96ea5df
Volume Id	fc98bbde-cf1d-4af1-bcfb-3368c96ea5df
Created At	Jun 03, 2025, 04:14 PM
Created By	valeriedelafuente@ucsb.edu
Updated At	Jun 03, 2025, 04:14 PM
Updated By	valeriedelafuente@ucsb.edu

databricks

Search data, notebooks, recents, and more...

⌘ + P

databricks-demo

Catalog

⚙️

🔄

+

32 GB, 4 Cores

Valerie De La Fuente's ...

Type to search...

🔍

▼

▼

My organization

▼

📁

databricks_demo

▼

📁

default

>

Tables (27)

>

Volumes (1)

📁

voterfile-export

▼

📁

information_schema

📁

catalog_privileges

📁

catalog_tags

📁

catalogs

📁

check_constraints

📁

column_masks

📁

column_tags

📁

columns

📁

constraint_column_usage

📁

constraint_table_usage

📁

information_schema_catalo...

📁

key_column_usage

Catalog Explorer

>

databricks_demo

>

default

>

📁

voterfile-export

📄

☆

⋮

Upload to this volume

Overview

Files

Details

Permissions

/Volumes/databricks_demo/default/vot...

🔍

Filter files and directori...

Name	Size	Last modified
📄 NJ_0000000000000.parquet	6.71 MB	4 days ago
📄 NJ_0000000000001.parquet	6.74 MB	4 days ago
📄 NJ_0000000000002.parquet	6.75 MB	4 days ago
📄 NJ_0000000000003.parquet	6.72 MB	4 days ago
📄 NJ_0000000000004.parquet	6.69 MB	4 days ago
📄 NJ_0000000000005.parquet	6.74 MB	4 days ago
📄 NJ_0000000000006.parquet	6.72 MB	4 days ago
📄 NJ_0000000000007.parquet	6.47 MB	4 days ago
📄 NJ_0000000000008.parquet	6.70 MB	4 days ago
📄 NJ_0000000000009.parquet	6.73 MB	4 days ago
📄 NJ_0000000000010.parquet	6.74 MB	4 days ago

2.2 Create a Databricks table from exported files in storage location.

The screenshot shows the Databricks Catalog Explorer interface. On the left, a sidebar displays a catalog tree with a search bar and a list of catalogs and tables. The 'nj_combined_all' table is selected. The main area shows the 'Overview' tab for this table, which includes a description, a table of columns with their types and comments, and a right sidebar with 'About this table' information. The 'About this table' section shows the owner as Valerie De La Fuente, the type as Managed, the data source as Delta, and the size as 1.4GiB, 52 files. The 'Tags' section has an 'Add tags' button, and the 'Row filter' section has an 'Add filter' button.

Column	Type	Comment	Tags	Column masking rule
SEQUENCE	bigint			
LALVOTERID	string			
Voters_Active	string			
Voters_StateVoterID	string			
Voters_CountyVoterID	string			
VoterTelephones_LandlineAreaCo...	string			
VoterTelephones_Landline7Digit	string			
VoterTelephones_LandlineFormat...	string			
VoterTelephones_LandlineUnform...	string			
VoterTelephones_LandlineConfid...	bigint			
VoterTelephones_CellPhoneOnly	string			
VoterTelephones_CellPhoneForm...	string			

2.3 EDA and Visualization with Databricks

1. Use at least 5 different Pyspark Pandas APIs to perform various data exploration tasks. Explain the documentation specification.

We are working with a large-scale voter file for New Jersey that has over 6.3 million rows and 441 columns. The data contains voter registration data, voting history, geolocation details, and consumer behavior attributes that all combine to create a profile for each voter. Each of the records, representing one individual voter, includes identifiers like `LALVOTERID`, sequence codes, and state and county voter IDs. Additionally, there is contact information and mailing addresses, confidence scores that reveal the data's reliability, detailed demographic information for each individual, cultural and linguistic attributes, and

property-related data. We are given an insight into household information like gender composition, family size, and political makeup. Some columns like `CountyEthnic_LALEthnicCode`, `Voters_PlaceOfBirth`, and `Community_College` have zero non-null values, or partial completeness like the `VoterTelephones_CellPhoneOnly` column, indicating areas where there is missing data. There are 47 integer-type columns, 2 float-type columns, and 392 string-type columns. Each row represents one voter record from a voter file.

6 APIs:

1st Pyspark Pandas API

▶

Just now (20s)

9

Python

✦

⌵

⋮

```
# 1st Pyspark Pandas API
# Preview of dataframe
df_head = df.head()
display(df_head)
```

▶ (2) Spark Jobs

Table ▾ +

🔍

🔧

📄

🗨

	¹ ₃ SEQUENCE	^A _C LALVOTERID	^A _C Voters_Active	^A _C Voters_StateVoterID	^A _C Voters_CountyVoterID	^A _C VoterTelephon
1	5293914	LALNJ2000120	A	M2326751989	0337307AD	856
2	5295089	LALNJ619791195	A	H2409761466	null	null
3	5306980	LALNJ3815577	A	L3417052717	0330451618	856
4	5307088	LALNJ429183	A	P6363951352	033014789	null
5	5307386	LALNJ2009174	A	M3313551992	0333230AD	856

⬇

5 rows | 19.87s runtime

Refreshed now

💡1

```
df_head = df.head()
display(df_head)
```

Pyspark documentation: `dataframe.head(n)` This function returns the first n rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it. If no variable is called it returns the first 5 rows.

2nd Pyspark Pandas API

▶

✓

05:28 PM (5m)

10

Python

✦

⌂

⋮

```
# 2nd Pyspark Pandas API
# Display column info using PySpark Pandas API
df_info = df.info()
display(df_info)
```

▶ (10) Spark Jobs

421	PresidentialPrimary_2004	242550	non-null	object
422	AnyElection_2003	1111681	non-null	object
423	General_2002	1277437	non-null	object
424	Primary_2002	259745	non-null	object
425	AnyElection_2001	1377177	non-null	object
426	General_2000	1770546	non-null	object
427	Primary_2000	366332	non-null	object
428	PresidentialPrimary_2000	366332	non-null	object
429	PRI_BLT_2020	1517126	non-null	object
430	PRI_BLT_2018	690288	non-null	object
431	PRI_BLT_2016	1294201	non-null	object
432	PRI_BLT_2014	358022	non-null	object
433	PRI_BLT_2012	480579	non-null	object
434	PRI_BLT_2010	353994	non-null	object
435	PRI_BLT_2008	398613	non-null	object
436	PRI_BLT_2006	250809	non-null	object
437	PRI_BLT_2004	242550	non-null	object
438	PRI_BLT_2002	259745	non-null	object
439	PRI_BLT_2000	366332	non-null	object
440	_rescued_data	0	non-null	object

dtypes: float64(2), int64(47), object(392)

```
df_info = df.info()
display(df_info)
```

Documentation Reference, PySpark Pandas `DataFrame.info()` : `df.info()` provides us with a concise summary of the DataFrame and helps better understand the structure of the dataset. This reveals the number of non-null entries per column, the total number of rows, memory usage, and the data type of each column.

3rd Pyspark Pandas API

```
▶ 05:40 PM (3s) 11 Python ✨ ⌵ ⋮

# 3rd Pyspark Pandas API
# 2012 voter parties: get unique values & corresponding counts
import warnings
warnings.filterwarnings('ignore')
df_unique_counts = df['PRI_BLT_2012'].value_counts()
display(df_unique_counts)

▶ (2) Spark Jobs

D    282101
R    198160
O         318
Name: PRI_BLT_2012, dtype: int64
```

```
import warnings
warnings.filterwarnings('ignore')
df_unique_counts = df['PRI_BLT_2012'].value_counts()
display(df_unique_counts)
```

Documentation Reference, PySpark Pandas `Series.value_counts()`: The API returns the frequency of unique values in the column `'PRI_BLT_2012'`, which details 2012 voter party affiliations. It counts the frequency of each unique value, sorts the output in descending order by frequency, and is helpful for analyzing class distributions and categorical variables.

4th Pyspark Pandas API

▶ ▼

✓ Just now (2s)

12

Python ✨ ⌵ ⋮

```
# 4th Pyspark Pandas API
# 2016 voter parties: get unique values & corresponding counts
import warnings
warnings.filterwarnings('ignore')
df_unique_counts = df['PRI_BLT_2016'].value_counts()
display(df_unique_counts)
```

▶ (2) Spark Jobs

D 857825
R 435481
O 895
Name: PRI_BLT_2016, dtype: int64

```
import warnings
warnings.filterwarnings('ignore')
df_unique_counts = df['PRI_BLT_2016'].value_counts()
display(df_unique_counts)
```

`.value_counts()` computes group sizes given the `groupby` command before, which in this case is grouping by people's vote for president in 2016. There are three parameters, `sort`, which is a boolean that if `true` sorts by frequencies, ascending, which if `true` sorts in ascending order, and `dropNA` which is `true` does not include counts of `NaN`.

5th Pyspark Pandas API

▶

✓

05:54 PM (5m)

13

Python

5th Pyspark Pandas API

Check for missing values in each column

df_missing = df.isnull().sum()

display(df_missing)

▶ (3) Spark Jobs

PresidentialPrimary_2004	6112158
AnyElection_2003	5243027
General_2002	5077271
Primary_2002	6094963
AnyElection_2001	4977531
General_2000	4584162
Primary_2000	5988376
PresidentialPrimary_2000	5988376
PRI_BLT_2020	4837582
PRI_BLT_2018	5664420
PRI_BLT_2016	5060507
PRI_BLT_2014	5996686
PRI_BLT_2012	5874129
PRI_BLT_2010	6000714
PRI_BLT_2008	5956095
PRI_BLT_2006	6103899
PRI_BLT_2004	6112158
PRI_BLT_2002	6094963
PRI_BLT_2000	5988376
_rescued_data	6354708

dtype: int64

```
df_missing = df.isnull().sum()
display(df_missing)
```

`df.isnull()` takes the dataframe and returns true for all the values in each column that are null. The `.sum()` at the end then sums all of the values from each column that return `True` so that we have a count of all of the missing values in the dataframe.

6th Pyspark Pandas API


```

import pyspark.pandas as ps
import pandas as pd
import plotly.express as px

# Step A: Years of interest
years = list(range(2000, 2020, 2)) # General elections every 2 years from 2000 to 2018

# Step B: Create voter turnout columns
for year in years:
    col_name = f'General_{year}'
    if col_name in df.columns:
        turnout_col = f'voter_turnout_{year}'
        df[turnout_col] = (df[col_name] == 'Y').astype(int)

# Step C: Select columns to keep
turnout_cols = [f'voter_turnout_{year}' for year in years if f'voter_turnout_{year}' in df.columns]
columns_needed = ['Voters_Age'] + turnout_cols
df_pd = df[columns_needed].dropna().to_pandas()

# Step D: Create age groups
bins = [20, 30, 40, 50, 60, 70, 80, 90]
labels = ['20s', '30s', '40s', '50s', '60s', '70s', '80+']
df_pd['age_group'] = pd.cut(df_pd['Voters_Age'], bins=bins, labels=labels, right=False)

# Step E: Melt to long format
df_melted = df_pd.melt(
    id_vars=['Voters_Age', 'age_group'],
    value_vars=turnout_cols,
    var_name='year',
    value_name='voter_turnout'
)

# Step F: Get numeric year
df_melted['year'] = df_melted['year'].str.extract(r'(\d{4})').astype(int)

# Step G: Get mean turnout

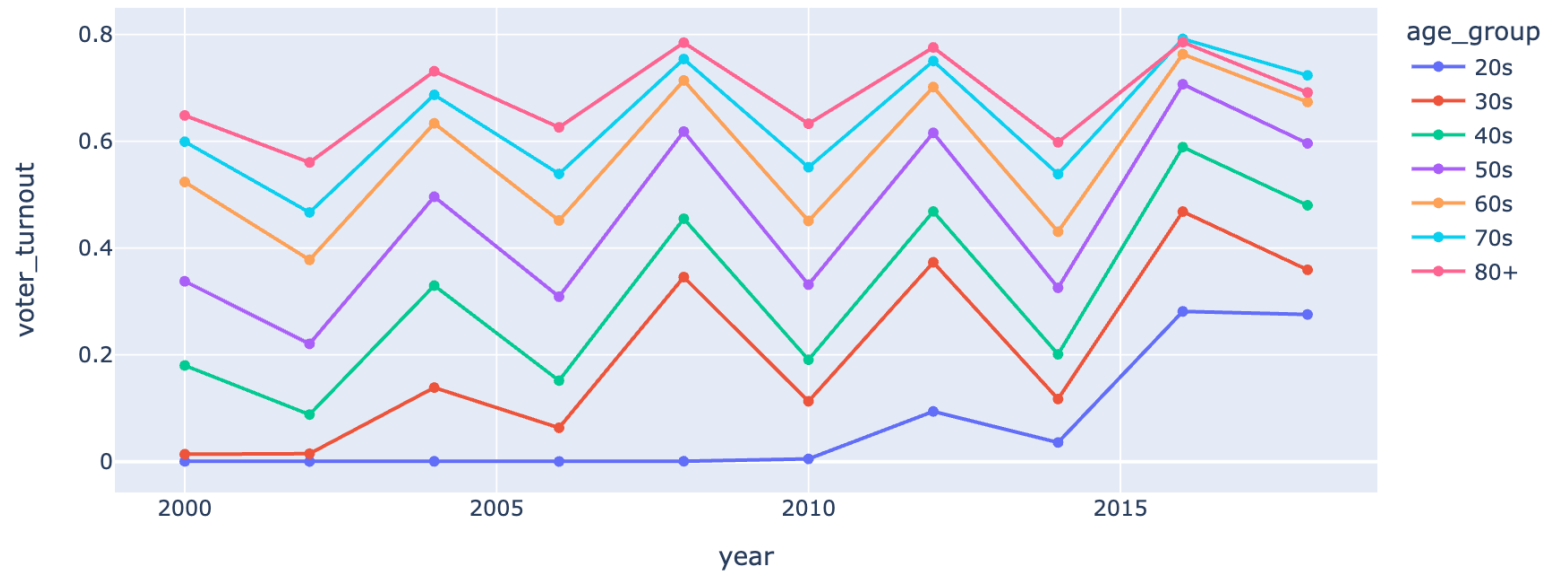
```



```
df_turnout = df_melted.groupby(['year', 'age_group'])  
['voter_turnout'].mean().reset_index()
```

```
# Step H: Plot  
fig = px.line(  
    df_turnout,  
    x='year',  
    y='voter_turnout',  
    color='age_group',  
    title='Voter Turnout by Age Group (2000–2018)',  
    markers=True  
)  
  
display(fig)
```

Voter Turnout by Age Group (2000–2018)



💡1

3. Create a choropleth visualization of voter turnout rate by county for a chosen year.

```
import pyspark.pandas as ps
import pandas as pd
import plotly.express as px
import requests
```

```
# Step A: Set-up
chosen_year = 2016
turnout_column = f"General_{chosen_year}"
```

```

# Create 5-digit FIPS
df = df.copy()
df['county_fips'] = df['Voters_FIPS'].apply(lambda x: f"34{x:03}")

# Step B: Remove null counties, de-duplicate by LALVOTERID
df_clean = df[df['County'].notna()].drop_duplicates(subset='LALVOTERID')

# Step C: Total registered voters by county
total_voters = (
    df_clean
    .groupby(['county_fips', 'County'])['LALVOTERID']
    .count()
    .reset_index()
    .rename(columns={'LALVOTERID': 'total_voters'})
)

# Step D: Actual voters (voted "Y")
voted_df = df_clean[df_clean[turnout_column] == 'Y']

actual_voters = (
    voted_df
    .groupby(['county_fips', 'County'])['LALVOTERID']
    .count()
    .reset_index()
    .rename(columns={'LALVOTERID': 'actual_voters'})
)

# Step E: Merge
turnout_grouped = ps.merge(total_voters, actual_voters, on=['county_fips', 'County'],
how='left')
turnout_grouped['actual_voters'] = turnout_grouped['actual_voters'].fillna(0)

# Step F: Get turnout rate
turnout_grouped['turnout_rate'] = turnout_grouped['actual_voters'] /
turnout_grouped['total_voters']

# Step G: convert to Pandas
turnout_grouped_pd = turnout_grouped.to_pandas()

```

```
# Step H: Load the GeoJSON
geojson_url = "https://raw.githubusercontent.com/plotly/datasets/master/geojson-counties-
fips.json"
geojson_data = requests.get(geojson_url).json()

# Step I: Plot choropleth
fig = px.choropleth(
    turnout_grouped_pd,
    geojson=geojson_data,
    locations="county_fips",
    color="turnout_rate",
    featureidkey="id",
    scope="usa",
    color_continuous_scale="Viridis",
    labels={"turnout_rate": "Turnout Rate"},
    title=f"Voter Turnout Rate by County in New Jersey ({chosen_year})",
    hover_data=["County", "turnout_rate"]
)

fig.update_geos(fitbounds="locations", visible=False)
fig.show()
```

Voter Turnout Rate by County in New Jersey (2016)



1

4. Create a choropleth visualization of party affiliation by county.

```
import pyspark.pandas as ps
import pandas as pd
import plotly.express as px
import requests

# Step A: Clean and prepare data
df = df.copy()
df = df[df['County'].notna() & df['Parties_Description'].notna()]
df['county_fips'] = df['Voters_FIPS'].apply(lambda x: f"34{x:03}")
```

```

# Step B: Count party affiliations per county
party_counts = (
    df.groupby(['county_fips', 'County', 'Parties_Description'])['LALVOTERID']
      .count()
      .reset_index()
      .rename(columns={
          'LALVOTERID': 'count',
          'County': 'county_name',
          'Parties_Description': 'party'
      })
)

# Step C: Pivot to wide format
pivot_psd = party_counts.pivot_table(
    index=['county_fips', 'county_name'],
    columns='party',
    values='count',
    fill_value=0
)

# Step D: Convert to pandas for computing idxmax
pivot_df = pivot_psd.to_pandas()
pivot_df['dominant_party'] = pivot_df.idxmax(axis=1)
pivot_df = pivot_df.reset_index()

# Step E: Load the GeoJSON
geojson_url = "https://raw.githubusercontent.com/plotly/datasets/master/geojson-counties-fips.json"
geojson_data = requests.get(geojson_url).json()

# Step F: Plot using Plotly
fig = px.choropleth(
    pivot_df,
    geojson=geojson_data,
    locations="county_fips",
    color="dominant_party",
    featureidkey="id",
    scope="usa",

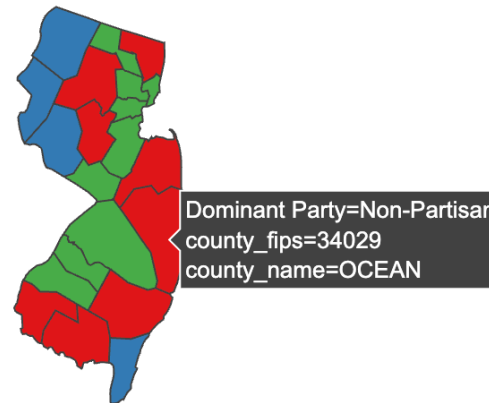
```

```
color_discrete_sequence=px.colors.qualitative.Set1,  
labels={"dominant_party": "Dominant Party"},  
title="New Jersey Party Affiliation by County",  
hover_data=["county_name"]  
)
```

```
fig.update_geos(fitbounds="locations", visible=False)  
fig.show()
```



New Jersey Party Affiliation by County



Dominant Party

- Non-Partisan
- Republican
- Democratic

1

3.3 Supervised and Unsupervised Learning

Research Question #1

How stable are individual party preferences over time, and what demographic or geographic patterns exist among voters who switch party affiliation between 2012 and 2016?

Supervised Learning Question: Can we predict whether a voter will switch party affiliation between 2012 and 2016 based on their demographics and prior party choices?

Definition:

Response variable: Binary switch indicator. 1 = voter changed from "D" to "R" or "R" to "D" between 2012 and 2016. 0 = voter stayed in the same party or consistently "U".

Loyal voter: One who had the same party in both 2012 and 2016.

Switcher: One who switched from "D" → "R" or "R" → "D".

Features (predictors): PRI_BLT_2012, PRI_BLT_2014 Age, gender, ethnicity County or FIPS code Turnout history (e.g., General_2012, General_2014)

Model (e.g., logistic regression, random forest) that identifies which voters are more likely to switch — useful for targeted persuasion in campaigns.

Use cross-validation to compare models

```
# Drop the null values

from pyspark.sql.functions import sum, when, col

# Convert to Spark DataFrame
sdf = df.to_spark()

# Get total number of columns
total_columns = len(sdf.columns)
```



```

# Compute non-null counts
non_null_counts = (
    sdf
    .agg(*[
        sum(when(col(c).isNotNull(), 1).otherwise(0)).alias(c)
        for c in sdf.columns
    ])
    .collect()[0]
    .asDict()
)

# Identify all-null columns
all_null_cols = [c for c, cnt in non_null_counts.items() if cnt == 0]
num_all_null = len(all_null_cols)

#print(f"Total columns    : {total_columns}")
#print(f"All-null columns: {num_all_null}")

# Drop all-null columns
sdf_cleaned = sdf.drop(*all_null_cols)

# Convert back to pyspark.pandas
df_cleaned = sdf_cleaned.pandas_api()

# Feature Engineering

import pyspark.pandas as ps
from pyspark.sql.functions import when, col
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer, OneHotEncoder, VectorAssembler
)
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

# Convert to Spark DataFrame
df_spark = df_cleaned.to_spark()

```

```

# Make switch-voter indicator
df_spark = df_spark.withColumn(
    "switched_party",
    when(
        ((col("PRI_BLT_2012") == "D") & (col("PRI_BLT_2016") == "R")) |
        ((col("PRI_BLT_2012") == "R") & (col("PRI_BLT_2016") == "D"))),
        1
    ).otherwise(0)
)

# Turnout flags
df_spark = df_spark.withColumn("turnout2012_flag", when(col("General_2012") == "Y",
1).otherwise(0)) \
    .withColumn("turnout2014_flag", when(col("General_2014") == "Y",
1).otherwise(0))

# Drop rows with nulls (BEFORE pipeline)
required_cols = [
    "PRI_BLT_2012", "PRI_BLT_2014", "Voters_Gender",
    "Ethnic_Description", "Voters_FIPS",
    "General_2012", "General_2014", "Voters_Age"
]
df_spark = df_spark.dropna(subset=required_cols)

# Define feature transformations
idx_2012      = StringIndexer(inputCol="PRI_BLT_2012",
outputCol="pri2012_index").setHandleInvalid("keep")
idx_2014      = StringIndexer(inputCol="PRI_BLT_2014",
outputCol="pri2014_index").setHandleInvalid("keep")
idx_gender    = StringIndexer(inputCol="Voters_Gender",
outputCol="gender_index").setHandleInvalid("keep")
idx_ethnicity = StringIndexer(inputCol="Ethnic_Description",
outputCol="ethnicity_index").setHandleInvalid("keep")
idx_fips      = StringIndexer(inputCol="Voters_FIPS",
outputCol="fips_index").setHandleInvalid("keep")

ohe = OneHotEncoder(
    inputCols=["pri2012_index", "pri2014_index", "gender_index", "ethnicity_index",

```

```

"fips_index"],
    outputCols=["pri2012_vec", "pri2014_vec", "gender_vec", "ethnicity_vec", "fips_vec"]
)

assembler = VectorAssembler(
    inputCols=[
        "pri2012_vec", "pri2014_vec", "gender_vec", "ethnicity_vec", "fips_vec",
        "turnout2012_flag", "turnout2014_flag", "Voters_Age"
    ],
    outputCol="features"
)

pipeline = Pipeline(stages=[idx_2012, idx_2014, idx_gender, idx_ethnicity, idx_fips, ohe,
assembler])

# Sample 5% of the data
df_sampled = df_spark.sample(fraction=0.05, seed=42)

# Fit the pipeline
pipeline_model = pipeline.fit(df_sampled)
prepared_sampled = pipeline_model.transform(df_sampled)

# Drop rows with null 'features' or 'label'
model_df = prepared_sampled.select(
    col("switched_party").alias("label"),
    col("features")
).dropna(subset=["features", "label"])

# Model Fitting and Results

# Train/test split
train_df, test_df = model_df.randomSplit([0.8, 0.2], seed=42)

# Train Logistic Regression
lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=10
)

```

```

lr_model = lr.fit(train_df)

# Evaluate model
predictions = lr_model.transform(test_df)
evaluator = BinaryClassificationEvaluator(
    labelCol="label", rawPredictionCol="rawPrediction", metricName="areaUnderROC"
)
auc = evaluator.evaluate(predictions)
accuracy = predictions.filter(col("prediction") == col("label")).count() /
predictions.count()
print(f"Test AUC = {auc:.4f}, Accuracy = {accuracy:.4f}")

# Hyperparameter tuning
paramGrid = (ParamGridBuilder()
    .addGrid(lr.regParam, [0.01, 0.1])
    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0])
    .build())

cv = CrossValidator(
    estimator=lr,
    estimatorParamMaps=paramGrid,
    evaluator=evaluator,
    numFolds=3
)
cv_model = cv.fit(train_df)
best_lr = cv_model.bestModel
#print("Best LR params:", best_lr.extractParamMap())

# Coefficients
#print("LR coefficients:", best_lr.coefficients)

# Analyze switch-voter demographics
switchers = prepared_sampled.filter(col("switched_party") == 1)
switchers.groupBy("ethnicity_index").count().orderBy("count", ascending=False).show(10)
switchers.groupBy("fips_index").count().orderBy("count", ascending=False).show(10)

```

Test AUC = 0.5946, Accuracy = 0.9892

ethnicity_index	count
2.0	25
0.0	18
4.0	15
3.0	12
5.0	8
6.0	6
1.0	5
9.0	3
11.0	2
7.0	1

only showing top 10 rows

fips_index	count
2.0	11
0.0	8
4.0	8
3.0	8
1.0	7
14.0	6
12.0	6
9.0	6
11.0	5
7.0	5

only showing top 10 rows

The AUC is equal to 0.5946. This is a low AUC, just slightly better than random guessing. This means the model is not very good at distinguishing between switchers and non-switchers. A high accuracy of 0.9892 is very high which is misleading in this context. This may be due to class imbalance (very few switchers vs. many non-switchers). It is interesting to note that `fips_index = 2.0` had the highest number of switchers (11 voters). If we wanted to further this analysis, we'd ideally join back the original `Voters_FIPS` column to each `fips_index`. Through the ethnicity metric, we found that certain

ethnicities featured more switchers than others. This suggests that certain groups in New Jersey may be more emotional, or more affected by hot-button issues in the more recent elections.

Research Question #2

What distinct voter behavior profiles can be identified based on demographic attributes and voting participation across multiple primary election cycles?

Rationale: Rather than assuming specific turnout behavior (e.g., consistent vs. inconsistent), this question seeks to uncover naturally occurring groupings (clusters) in the voter population. These groupings may reflect patterns such as:

- Demographic clusters with sporadic vs. reliable turnout
- Voters who only engage in presidential primary years
- Clusters defined by age, ethnicity, location, and voting frequency
- This approach is valuable for exploratory analysis and can inform targeted messaging, resource allocation, or hypothesis generation for future predictive models.

Unsupervised Learning Question: Can we identify distinct clusters of voters based on their multi-year primary election participation patterns and demographic attributes?

- Variables to use in clustering: voting behavior, age group, ethnicity, gender, county or ZIP-level indicator
- Model: K-Means

```
# Data Reprocessing and Feature Engineering
from pyspark.sql.functions import col, when
from pyspark.ml.feature import StringIndexer, VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
from pyspark.ml import Pipeline

# Define relevant columns
election_cols = [f"Primary_{year}" for year in range(2000, 2022, 2)]
```

```

demographic_cols = [
    "age_group",
    "Ethnic_Description",
    "Voters_Gender",
    "Residence_Addresses_Zip"
]
all_cols = demographic_cols + election_cols
df_spark = df_cleaned.to_spark()

from pyspark.sql.functions import when

df_spark = df_spark.withColumn(
    "age_group",
    when(col("Voters_Age") < 30, "Under 30")
    .when((col("Voters_Age") >= 30) & (col("Voters_Age") < 50), "30-49")
    .when((col("Voters_Age") >= 50) & (col("Voters_Age") < 70), "50-69")
    .otherwise("70+")
)

# Subset and drop rows with missing data
df_subset = df_spark.select(*all_cols).dropna(subset=all_cols)

# Convert "Y"/"N" to 1/0 for primary columns
for col_name in election_cols:
    df_subset = df_subset.withColumn(
        col_name, when(col(col_name) == "Y", 1).otherwise(0)
    )

# Encode categorical variables
indexers = [
    StringIndexer(inputCol="age_group", outputCol="age_group_idx", handleInvalid="keep"),
    StringIndexer(inputCol="Ethnic_Description", outputCol="ethnicity_idx",
handleInvalid="keep"),
    StringIndexer(inputCol="Voters_Gender", outputCol="gender_idx",
handleInvalid="keep"),
    StringIndexer(inputCol="Residence_Addresses_Zip", outputCol="zip_idx",

```

```

handleInvalid="keep")
]

# Assemble features
assembler = VectorAssembler(
    inputCols=[
        "age_group_idx", "ethnicity_idx", "gender_idx", "zip_idx"
    ] + election_cols,
    outputCol="features_assembled"
)

# Standardize features
scaler = StandardScaler(inputCol="features_assembled", outputCol="features")

# Model Fitting and Results
# KMeans Clustering
kmeans = KMeans(featuresCol="features", predictionCol="cluster", k=4, seed=42)

# Build and run pipeline
pipeline = Pipeline(stages=indexers + [assembler, scaler, kmeans])
model = pipeline.fit(df_subset)
clustered_df = model.transform(df_subset)

# Cluster results
#clustered_df.select(
#    *demographic_cols, *election_cols, "cluster"
#).show(20)

# Demographic Information by Cluster
pdf = clustered_df.toPandas()

# Calculate average turnout rates
turnout_summary = pdf.groupby("cluster")[["Primary_2008", "Primary_2012",
"Primary_2016"]].mean()

# Count distributions by cluster
age_counts = pdf.groupby(["cluster", "age_group"]).size().unstack(fill_value=0)
ethnicity_counts = pdf.groupby(["cluster",

```



```

"Ethnic_Description"]].size().unstack(fill_value=0)
gender_counts = pdf.groupby(["cluster", "Voters_Gender"]).size().unstack(fill_value=0)

# Combine all summaries into one DataFrame
cluster_profile = pd.concat([turnout_summary, age_counts, ethnicity_counts,
gender_counts], axis=1)

# Add cluster number as a column instead of just index
cluster_profile.reset_index(inplace=True)

# Display the table
display(cluster_profile)

import plotly.express as px
import pandas as pd

# Expanded ZIP centroid lookup
zip_centroids = pd.DataFrame({
    "Residence_Addresses_Zip": ['07002', '07047', '07093', '07103', '07112', '08542',
'08750', '07860'],
    "latitude": [40.65, 40.79, 40.80, 40.74, 40.69, 40.35, 40.13, 41.21],
    "longitude": [-74.13, -74.01, -74.02, -74.19, -74.21, -74.66, -74.04, -74.63]
})

# Include a Cluster 2 ZIP
zip_cluster_df = pd.DataFrame({
    "Residence_Addresses_Zip": ['07002', '07047', '07093', '07103', '07112', '08542',
'08750', '07860'],
    "cluster": [3, 1, 1, 1, 1, 0, 0, 2] # <-- 07860 assigned to cluster 2
})

# Merge on ZIP to get coordinates
zip_map_df = pd.merge(zip_cluster_df, zip_centroids, on="Residence_Addresses_Zip",
how="left")

# Create scatter map
fig = px.scatter_mapbox(
    zip_map_df,
    lat="latitude",

```

```

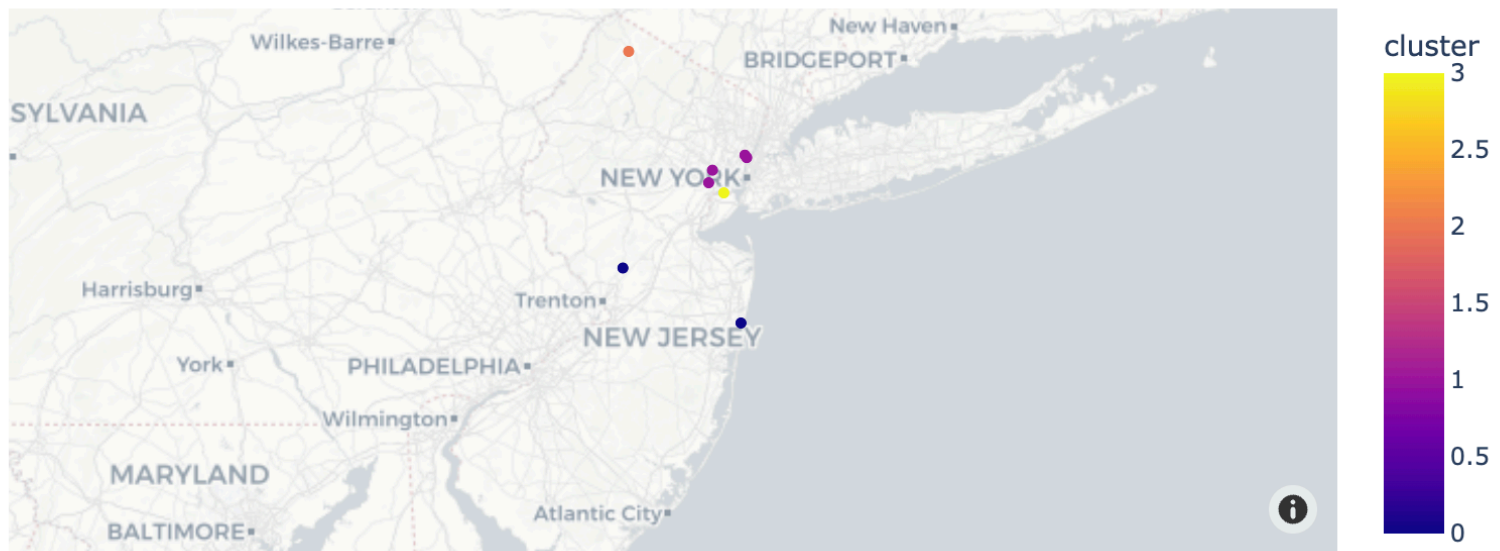
lon="longitude",
color="cluster",
hover_name="Residence_Addresses_Zip",
zoom=7,
mapbox_style="carto-positron",
title="ZIP Code Cluster Locations"
)

fig.show()

```

Table ▼ + 🔍 🔧 📄							
	1 ² ₃ cluster	1.2 Primary_2008	1.2 Primary_2012	1.2 Primary_2016	1 ² ₃ 30–49	1 ² ₃ 50–69	1 ² ₃ 70+
1	0	1	1	1	4	617	1444
2	1	1	1	1	0	0	3998
3	2	1	1	1	0	0	3131
4	3	1	1	1	250	3523	0

ZIP Code Cluster Locations



After clustering the data into four distinct groups, we found notable similarities and differences among them. All of the groups had 100% voter turnout, which is likely because of how we thoroughly dropped null values due to prior issues with the code. The vast majority of voters in our dataset were over 70, with cluster 3 to be the only group to not have any. Additionally, cluster 1 was specifically female and cluster 2 was male. The most interesting distinction we found from our four clusters was by location. Cluster 0 was the most dispersed and represented a lot of South Jersey, particularly the smaller shore towns and areas by Trenton and Camden. Cluster 1 represented the suburban areas in North Jersey, especially Hudson and Essex Counties near New York City. Cluster 2 shows a lot of overlap, particularly North West New Jersey which is also a more sparse population. Cluster 3 represents the most dense population areas in New Jersey, like Jersey City and Newark, which are even closer to New York City.